



Scenario 23 - Dots and Boxes

Overview

Dots and Boxes is a turn-based grid game. Players draw one line at a time between adjacent dots. Completing the fourth side of a box claims that box, earns a point, and gives the same player another turn.

The starter project is deliberately split into separate modules for game flow, board state, and rule validation. Read the existing implementation carefully before changing it.

How to run

From this folder:

```
python main.py
```

Enter moves in the form:

H row column

or

V row column

Rows and columns start at zero.

Task 1 — Reproduce and investigate the bug

Run several games and deliberately create situations where a move completes one or more boxes.

Compare the score and turn behaviour before and after box completion. Identify the incorrect behaviour, trace it through the board and game-state logic, and fix it without replacing the modular structure.

Your before-change video should capture the broken behaviour clearly.

Task 2 — Add a meaningful feature

Add a substantial gameplay feature that requires changes across

more than one module.

The feature should make the game more complete rather than simply changing text or appearance. It should interact correctly with the existing board state and turn/score system.

Task 3 — Validation and robustness

Strengthen input and game-state handling.

The program should safely handle malformed commands, invalid coordinates, repeated lines, and moves made after the board is already complete. Invalid input must not corrupt the board or score.

Task 4 — Testing and quality

Create or expand automated tests covering the important game rules.

Include tests for at least:

- a valid horizontal move
- a valid vertical move
- an invalid/repeated move
- completion of a box
- the end-of-game condition

Document the changes you made and any design decisions that were important to the solution.

Constraints

Keep the project modular.

Do not replace the game with an unrelated implementation.

Preserve the existing gameplay.

Avoid putting all new logic into main.py.

Keep third-party dependencies out unless there is a clear need for them.

Submission Checklist

Submission is only the following three things:

- A 10-second video of gameplay before your changes, showing the bug/broken behavior

A 10-second video of gameplay after your changes, showing the bug fixed and the new features working

The Chat/LLM used page link, with the complete chat history
this is the readme file of my topic see the pdf and guide me to complete the task see the deliverables properly

Yes. I checked the **Lab 4 Student Handout PDF** and compared it with your **Scenario 23 – Dots and Boxes README**.

The important point is: **your final submission has 3 deliverables in the README, but the PDF additionally tells you exactly how the work should be done in GitHub.**  Lab_4_VibeCoding_Student_handout

Your exact deliverables

#	What you need	What it should contain
1	Before video – 10 sec	Show the original/broken game and clearly reproduce the box-completion bug
2	After video – 10 sec	Show the bug fixed + new feature working
3	Chat/LLM history	Complete ChatGPT conversation used to do the assignment

And according to the PDF, your Lab-4 GitHub folder should contain **Videos, Updated Code, and Chat history exported as a document/PDF.**  Lab_4_VibeCoding_Student_handout

There is also an important GitHub requirement

The PDF says you must:

- clone/fork the assigned repo
- run the Python game
- record the **before** video
- use prompts to fix/add features
- record the **after** video
- **push the code to your personal repo**
- make **separate commits for each task**
- **do not create a PR to the main SETAPESU26 repo.**  Lab_4_VibeCoding_Student_handout

So your work should look roughly like this:

</> Plain text

Your Personal GitHub Repository

```
|
├── Lab-4
│   ├── Videos
│   │   ├── Before.mp4
│   │   └── After.mp4
│   ├── Updated_Code
│   │   ├── Scenario-23-Dots-and-Boxes
│   │   │   ├── main.py
│   │   │   └── ...other project files...
│   └── Chat_History
│       └── LLM_Conversation.pdf
```

Do not create the Lab-4 folder yet if you haven't finished the code/video/chat export. We can create/upload it at the end.

Your four tasks

We should complete them **one at a time**, because the handout explicitly says to write prompts for each feature one by one and make separate commits.  Lab_4_VibeCoding_Student_handout

1. Task 1 — Bug fix

- Reproduce the broken score/turn behaviour.
- Record the **10-second BEFORE video before changing anything**.
- Investigate the board/game-state logic.
- Fix it without destroying the modular structure.
- Commit Task 1 separately.

2. Task 2 — Meaningful feature

- Add a real gameplay feature.
- It must involve more than one module.
- It must interact with board state and score/turn handling.
- Commit Task 2 separately.

3. Task 3 — Validation

- malformed commands
- invalid coordinates
- repeated lines
- moves after game completion
- invalid input must not alter score/board.
- Commit Task 3 separately.

4. Task 4 — Automated tests

- horizontal move
- vertical move

- invalid/repeated move
- box completion
- end of game
- document important design decisions.
- Commit Task 4 separately.

Most important thing right now

Before we modify anything, we need to make sure your original project is still in its original/broken state and reproduce Task 1.

Then we record the **10-second BEFORE video**.

After that we can safely start Task 1 and proceed through Tasks 2–4.

Also, your README says the final submission is **only the two 10-second videos + Chat/LLM page link**, while the college handout says the **Lab-4 repo should contain Videos + Updated Code + Chat history PDF**. So to be safe, **we should satisfy both**: upload all three categories to your Lab-4 GitHub folder, while your actual assignment submission contains the requested three items.

If you want, **we can now continue exactly from where you stopped and do Task 1 → BEFORE VIDEO first**, without changing any code yet.

STEP 1

STEP 1 — Reproduce the original bug

Do not change any code yet. We first need the **BEFORE video** showing the broken behavior.

You already have the exact bug scenario from your README:

Plain text

```
H 0 0
H 0 1
V 0 0
V 0 2
V 0 1
```

Do this now

1. Open your Dots and Boxes project folder in VS Code.

2. Open the VS Code terminal.

Make sure you are inside the folder containing `main.py`.

Run:

 Bash

```
python main.py
```

3. Enter these moves one by one:

 Plain text

```
H 0 0  
H 0 1  
V 0 0  
V 0 2  
V 0 1
```

The last move **V 0 1** completes two boxes at once.

What we are checking

At the end, look at:

 Plain text

```
Scores  
Turn
```

The original bug should show incorrect turn/score behavior after the two boxes are completed.

Do not fix anything yet.

STEP 1A — Record the BEFORE video

Your college handout specifically says to record **10 seconds before making any changes**.

 Lab_4_VibeCoding_Student_handout

For the 10-second recording, show:

1. The terminal/game running.
2. Enter the moves above.
3. Show the final result where the **incorrect behavior is visible**.

Save it as something simple like:

 Plain text

Before.mp4

⚠ Do not upload/push this video to GitHub yet.

After you run those 5 moves, send me a screenshot of the final terminal output. Then I'll tell you exactly what we do for Step 2: investigate and fix Task 1.

STEP 2

STEP 2 — Investigate and fix Task 1

Now we move from **reproducing the bug** to finding where it happens.

Do not add Task 2's feature yet. We should finish Task 1 completely first.

2.1 Open the project files

In VS Code, look at the project structure. Because your README says the project is deliberately separated into modules for:

- game flow
- board state
- rule validation

we need to inspect those existing modules rather than putting the fix into `main.py`.

2.2 Use this ChatGPT prompt

Since this assignment is specifically about VibeCoding, use a prompt like this in your LLM chat:

Task 1 Investigation Prompt

① Original

② Write Like Me



I am working on Scenario 23 – Dots and Boxes for my Lab 4 VibeCoding assignment.

I have reproduced the Task 1 bug using these moves:

H 0 0

H 0 1

V 0 0

V 0 2

V 0 1

The final move V 0 1 completes two boxes at once. The game incorrectly handles the score/turn behaviour after box completion.

Please first inspect the existing project files and explain:

1. Which module stores the board state?

2. Which module checks whether a box is completed?
3. Which module controls score and player turns?
4. Trace what happens when V 0 1 completes two boxes.
5. Identify the exact cause of the incorrect score/turn behaviour.

Do not rewrite the project or change its modular structure. After explaining the cause, propose the smallest appropriate fix.

2.3 Important

Don't immediately accept a generated rewrite.

The assignment specifically says to **trace the problem through the board and game-state logic and fix it without replacing the modular structure.**

So we want to understand:

`</>` Plain text

```
User move
  ↓
Input/game flow
  ↓
Board state updated
  ↓
Check completed boxes
  ↓
Score updated
  ↓
Turn decided
```

The important case is:

`</>` Plain text

```
V 0 1
  ↓
2 boxes completed
  ↓
score should increase by 2
  ↓
same player gets another turn
```

2.4 After the LLM identifies the bug

Don't make any other changes.

Send me the LLM's response showing the **identified bug and proposed fix** (or screenshots of it).

I'll check it with you before you apply the fix.

Then we'll do:

STEP 3 → Apply Task 1 fix → test it → make the Task 1 Git commit.

Here is the step-by-step summary of what you need to do to complete Task 1:

1. File Responsibilities Summary

- **board.py (Board State Storage):** Manages line states (`self.horizontal` , `self.vertical`), tracks completed coordinates (`self.completed`), and displays the board layout.
- **rules.py (Box Completion Logic):** Calculates `completed_boxes(board, before)` by computing `len(board.completed - before)` .
- **game.py (Score & Turn Control):** Runs the main loop, updates `self.scores[self.current]` by `newly_completed` , and switches turns (`self.current = 1 - self.current`) only when `newly_completed == 0` .

2. Implementation Steps

Step 1: Update `board.py`

Modify `_update_completed()` inside `board.py` to prevent re-evaluating already completed boxes and ensure new boxes are clean additions:

Python

```
def _update_completed(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (r, c) not in self.completed:
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
```

```

        and self.vertical[r][c]
        and self.vertical[r][c + 1]
    ):
        self.completed.add((r, c))

```

Step 2: Confirm `game.py` Logic

Ensure the scoring section in `game.py` correctly adds `newly_completed` to `self.scores` and keeps the active turn:

Python

```

before = set(self.board.completed)
self.board.add_line(orientation, row, col)
newly_completed = completed_boxes(self.board, before)

if newly_completed:
    self.scores[self.current] += newly_completed
    print(f"Player {self.current + 1} completed {newly_completed} box(es).")
else:
    self.current = 1 - self.current

```

3. Verification

Run your program and input the test move sequence:

1. H 0 0
2. H 0 1
3. V 0 0
4. V 0 2
5. V 0 1

Expected Behavior:

- Move V 0 1 completes **2 boxes** simultaneously.
- Player 1 receives **+2 points** in `Scores`.
- Player 1 keeps the turn ("**Player 1 completed 2 box(es) and plays again.**").
- Both boxes display an `x` inside them on the board.

Yes. This is the **Task 1 investigation result** you got from the LLM. Before changing anything, there is one important thing: **we should verify that this proposed fix matches your actual code** rather than blindly pasting it.

STEP 2A — Check the actual files

Please open these three files in VS Code:

</> Plain text

```
board.py
rules.py
game.py
```

Then **send me the contents of all three files** (you can copy-paste them here).

I will check:

- whether `_update_completed()` currently has the bug described,
- whether `completed_boxes()` is already correct,
- whether `game.py` already handles `newly_completed` correctly,
- and exactly **which lines need to be changed**.

⚠ **Do not modify `board.py` yet.**

Once I verify your actual code, I'll guide you through the change step-by-step, then we'll run the five test moves and confirm:

</> Plain text

```
Player 1 completed 2 box(es) and plays again.
Scores: P1=2 P2=0
Turn: P1
```

That will complete **Task 1's code fix** before we move to Task 2.

```
class Board:
    def __init__(self, rows=2, cols=2):
        self.rows = rows
        self.cols = cols
        self.horizontal = [[False] * cols for _ in range(rows + 1)]
        self.vertical = [[False] * (cols + 1) for _ in range(rows)]
        self.completed = set()
```

```

def add_line(self, orientation, row, col):
    if orientation == "H":
        self.horizontal[row][col] = True
    else:
        self.vertical[row][col] = True
    self._update_completed()

def _update_completed(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (
                self.horizontal[r][c]
                and self.horizontal[r + 1][c]
                and self.vertical[r][c]
                and self.vertical[r][c + 1]
            ):
                self.completed.add((r, c))

def is_complete(self):
    total = self.rows * (self.cols + 1) + self.cols * (self.rows + 1)
    used = sum(map(sum, self.horizontal)) + sum(map(sum,
self.vertical))
    return used == total

def display(self, scores, current):
    print()
    print(f"Scores: P1={scores[0]} P2={scores[1]} | Turn: P{current
+ 1}")

    for r in range(self.rows + 1):
        print(".".join("----" if self.horizontal[r][c] else " " for c in
range(self.cols)))
        if r < self.rows:
            middle = []
            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)
            if c < self.cols:
                middle.append(" " + ("X" if (r, c) in self.completed
else " ") + " ")

```

```
print("".join(middle))
print() board board.py
```

Yes — I checked your **actual board.py** .

What I found

Your current `_update_completed()` is:

 Python

```
def _update_completed(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (
                self.horizontal[r][c]
                and self.horizontal[r + 1][c]
                and self.vertical[r][c]
                and self.vertical[r][c + 1]
            ):
                self.completed.add((r, c))
```

This **does not check whether** `(r, c)` **is already completed.**

So the proposed Task 1 change is valid for your actual file.

STEP 2B — Change only this part

Replace your current `_update_completed()` with:

 Python

```
def _update_completed(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (r, c) not in self.completed:
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
                    and self.vertical[r][c]
                    and self.vertical[r][c + 1]
                ):
                    self.completed.add((r, c))
```

Do not change anything else in board.py .

Why?

Previously, every call to `_update_completed()` checked **all boxes again.**

Now it checks only boxes that are **not already in `self.completed`** .

This makes newly completed boxes cleanly distinguishable when `game.py` compares:

 Python

```
before = set(self.board.completed)
```

with the board state after the move.

Next

After you make this **one change and save `board.py`** , don't change anything else.

Then send me your **`rules.py`** and **`game.py`** contents. I'll verify those two files before you run the test.

```
from board import Board
from rules import valid_move, completed_boxes

class DotsAndBoxes:
    def __init__(self):
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]

    def run(self):
        print("Dots and Boxes")
        print("Enter moves as H row col or V row col.")
        print("Rows and columns start at 0.")
        print("Example: H 0 1")

        while not self.board.is_complete():
            self.board.display(self.scores, self.current)
            raw = input(f"Player {self.current + 1}, move: ")
            raw = raw.strip().upper()
            parts = raw.split()

            if len(parts) != 3:
                print("Invalid format.")
                continue
```

```

orientation, row, col = parts
if not row.isdigit() or not col.isdigit():
    print("Row and column must be numbers.")
    continue

row, col = int(row), int(col)
if not valid_move(self.board, orientation, row, col):
    print("Invalid or already-used move.")
    continue

before = set(self.board.completed)
self.board.add_line(orientation, row, col)
newly_completed = completed_boxes(self.board, before)

if newly_completed:
    self.scores[self.current] += newly_completed
    print(f"Player {self.current + 1} completed
{newly_completed} box(es) and plays again.")
else:
    self.current = 1 - self.current

self.board.display(self.scores, self.current)
print("Game over!")
if self.scores[0] == self.scores[1]:
    print("The game is a draw.")
else:
    winner = 1 if self.scores[0] > self.scores[1] else 2
    print(f"Player {winner} wins!") this is in repo game .py

```

Yes, I checked your actual `game.py` . **For Task 1, your `game.py` already has the correct score/turn logic. Do not change it.**

The important section is already:

</> Python

```

if newly_completed:
    self.scores[self.current] += newly_completed
    print(f"Player {self.current + 1} completed {newly_completed} box(es) and plays again.")
else:
    self.current = 1 - self.current

```

That means:

- If 1 **box** is completed → player gets +1 and plays again.
- If 2 **boxes** are completed → player gets +2 and plays again.
- If 0 **boxes** are completed → turn changes.

So our Task 1 change is **only** `board.py` .

What you should have now

`board.py` — changed

 Python

```
def _update_completed(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (r, c) not in self.completed:
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
                    and self.vertical[r][c]
                    and self.vertical[r][c + 1]
                ):
                    self.completed.add((r, c))
```

`game.py` — unchanged 

Before we run it

I still need to see your `rules.py` because `game.py` uses:

 Python

```
valid_move
completed_boxes
```

We need to make sure those functions are compatible with the fix.

Send your `rules.py` next. Don't change it yet.

```
def valid_move(board, orientation, row, col):
    if orientation not in {"H", "V"}:
        return False

    if orientation == "H":
        return (
```

```

    0 <= row <= board.rows
    and 0 <= col < board.cols
    and not board.horizontal[row][col]
)

return (
    0 <= row < board.rows
    and 0 <= col <= board.cols
    and not board.vertical[row][col]
)

def completed_boxes(board, before):
    return len(board.completed - before) this is rule rules .py

```

Perfect. I checked your actual `rules.py` , and for **Task 1**, it is already correct. 

Your three modules now

File	Task 1 status
<code>board.py</code>	 Change needed — we added the <code>not in self.completed</code> check
<code>rules.py</code>	 No change needed
<code>game.py</code>	 No change needed

Your `completed_boxes()` already does exactly what `game.py` needs:

 Python

```
return len(board.completed - before)
```

So we should **not modify** `rules.py` .

Now STEP 2 is complete

The Task 1 fix is only this:

 Python

```
if (r, c) not in self.completed:
```

inside `_update_completed()` .

This keeps your existing modular structure, which is exactly what the assignment requires.

STEP 3 — Test the Task 1 fix

Now save `board.py` .

Then run:

`</>` Bash

```
python main.py
```

Enter these **exactly in this order**:

`</>` Plain text

```
H 0 0
H 0 1
V 0 0
V 0 2
V 0 1
```

The final move:

`</>` Plain text

```
V 0 1
```

should complete **2 boxes**.

Expected result

You should see something similar to:

`</>` Plain text

```
Player 1 completed 2 box(es) and plays again.
```

and:

`</>` Plain text

Scores: P1=2 P2=0 | Turn: P1

You should also see **two x marks** on the completed boxes.

Important

If this works, **STOP** there.

Don't start Task 2 yet.

Send me a **screenshot of the final output**, and I'll guide you through **STEP 4 — verify Task 1, make the separate Task 1 Git commit, and then move to Task 2.**

[NEXT](#)

STEP 4 — Finish Task 1 and make the separate Git commit

If your test worked and you saw **Player 1 = 2 points** and **Player 1 gets another turn**, Task 1 is complete. 

4.1 Check the changed file

Only this file should have been modified for Task 1:

 Plain text

```
board.py
```

`game.py` and `rules.py` should remain unchanged.

4.2 Check Git status

In your VS Code terminal, run:

 Bash

```
git status
```

You should see `board.py` under **modified files**.

4.3 Review the change

Run:

 Bash

```
git diff
```

You should see the addition around:

 Python

```
if (r, c) not in self.completed:
```

4.4 Make the Task 1 commit

Run:

 Bash

```
git add board.py
```

Then:

 Bash

```
git commit -m "Fix box completion and turn handling"
```

Then:

 Bash

```
git push
```

Important assignment requirement

The college handout specifically requires **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

So do not combine Task 1, Task 2, Task 3, and Task 4 into one commit.

Your Git history should eventually look roughly like:

 Plain text

```
Task 1 commit → Bug fix  
Task 2 commit → New gameplay feature  
Task 3 commit → Validation/robustness  
Task 4 commit → Automated tests
```

After Task 1 commit

Do not start Task 2 by yourself yet.

Next we need to choose and implement the **meaningful gameplay feature** required by your Scenario 23 README, while keeping it modular and changing more than one module.

Once your Task 1 commit is pushed, tell me **"done"**, and we'll start **STEP 5 — Task 2**.

TASK 2

STEP 5 — Task 2: Add a meaningful gameplay feature

Your README does **not specify one particular feature** for Task 2. It says the feature must:

- make the game more complete,
- require changes across **more than one module**,
- interact correctly with the existing **board state and turn/score system**,
- and not just change text or appearance.

So we should choose a feature that is substantial but still safe to implement within your existing modular design.

I recommend: Undo Last Move

This is a good fit for Dots and Boxes because it affects:

- **Board state** — lines and completed boxes need to be restored.
- **Game state** — score and current player's turn need to be restored.
- **Game flow/input** — the player needs a way to request Undo.

That satisfies the requirement that Task 2 changes more than one module.

But don't modify code yet

First, use your LLM to generate the implementation plan. Since your assignment is specifically VibeCoding and expects prompts for the features one by one, use this prompt:

Task 2 Prompt

① Original

② Write Like Me



I am continuing Scenario 23 – Dots and Boxes for my Lab 4 VibeCoding assignment.

Task 1 is already completed and committed. The box-completion bug was fixed in board.py. Do not modify or undo the Task 1 fix.

For Task 2, I need to add a meaningful gameplay feature. I want to add an "Undo Last Move" feature.

Requirements:

1. The player should be able to undo the most recent valid move.
2. Undo must restore the board line state.
3. Undo must restore any boxes completed by that move.

4. Undo must restore the score to its previous value.
5. Undo must restore the correct player's turn.
6. Invalid moves should not create undo history.
7. Undo should work for a move that completed one or more boxes.
8. Undo should work without breaking the existing Task 1 behavior.
9. The feature must involve more than one module and preserve the existing modular structure.
10. Do not put all new logic into main.py or rewrite the project.

First inspect the existing board.py, rules.py, and game.py and explain which modules need to change and why. Then propose the smallest clean implementation. Do not make unrelated changes.

What we want from the LLM

It should identify something similar to:

</> Plain text

game.py

↓

handles Undo command + saves/restores score/turn

board.py

↓

handles restoring the previous board state

rules.py

↓

existing validation/rule logic remains reusable

Do not paste the generated code yet.

Send me the LLM's response after you use that prompt. I'll check the proposed Task 2 design against your actual project before you change anything.

Module Responsibilities & Changes Required

To implement the **Undo Last Move** feature cleanly while maintaining the existing modular architecture, changes are required across two modules: **board.py** and **game.py**.

- **board.py (Board State Management):**
 - **Why change:** Board encapsulates the line grid arrays (horizontal and vertical) and the completed box

set. It needs a method to remove a line and re-evaluate or restore the set of completed boxes.

- **What to add:** A `remove_line(orientation, row, col)` method that sets the corresponding grid line back to `False` and removes any boxes from `self.completed` that are no longer enclosed by 4 valid lines.
- **game.py (Move History, Scores & Turn Control):**
 - **Why change:** `DotsAndBoxes` controls the user interface input loop, score updates, turn toggling, and move sequence history.
 - **What to add:**
 - A move history stack (`self.history`) that stores snapshot state records for valid moves: `(orientation, row, col, player, points_scored)` .
 - Recognition of the command "UNDO" (or "U").
 - Undo execution logic that pops the last record, removes the line via `board.remove_line(...)` , deducts `points_scored` from that player's score, and restores `self.current` to the player who made that move.
- **rules.py (No Changes Required):**
 - `rules.py` only contains `valid_move` and `completed_boxes` . The existing functions operate strictly on the current board state and require no modifications.

2. Implementation Plan

Step 1: Update `board.py`

Add `remove_line` to `Board` in `board.py` :

Python

```
def remove_line(self, orientation, row, col):
    if orientation == "H":
        self.horizontal[row][col] = False
    else:
        self.vertical[row][col] = False
```

```
# Recompute completed boxes based on current
valid_completed = set()
for r in range(self.rows):
    for c in range(self.cols):
        if (
            self.horizontal[r][c]
            and self.horizontal[r + 1][c]
            and self.vertical[r][c]
```

Step 2: Update `game.py`

In `game.py` :

1. Initialize `self.history = []` in `__init__` .
2. Add support for "UNDO" in the input handling loop.
3. Push valid moves to `self.history` **before** or **during** move processing.
4. Revert state on "UNDO" .

Python

```
from board import Board
from rules import valid_move, completed_boxes

class DotsAndBoxes:
    def __init__(self):
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]
        self.history = [] # Stack of (orientation,

    def run(self):
        print("Dots and Boxes")
        print("Enter moves as H row col or V row co
        print("Rows and columns start at 0.")
```

3. Verification & Test Walkthrough

1. Basic Undo Test:

- Make a valid move: H 0 0 .
- Type UNDO .
- Board line is removed and Player 1 turn is restored.

2. Scoring & Double-Box Undo Test:

- Reproduce Task 1 double-completion sequence: H 0 0 , H 0 1 , V 0 0 , V 0 2 , V 0 1 .
- Player 1 completes 2 boxes and gains 2 points (Scores: P1=2 P2=0).
- Type UNDO .
- Move V 0 1 is removed, both completed boxes (X) are cleared from self.board.completed , P1's score reverts to 0 , and turn remains with Player 1.

3. Invalid Input Test:

- Type an invalid move like H 99 99 .
- Program reports Invalid or already-used move.
- Type UNDO .
- The previously played valid move is undone, ignoring the invalid input attempt.

Yes. The proposed **Task 2 design** is appropriate for your **README requirements**, and it changes two modules (board.py and game.py) while keeping the project modular.

But there is **one important correction before you paste the code**: the suggested game.py records history **before** the score/turn update, which is okay for score restoration, but we should carefully test the turn behavior when undoing a move that did **not** complete a box.

STEP 5A — Apply Task 2

We will make only these changes:

1. board.py

Add this method inside the Board class:

Python

```
def remove_line(self, orientation, row, col):
    if orientation == "H":
        self.horizontal[row][col] = False
    else:
        self.vertical[row][col] = False

    valid_completed = set()
```

```

    for r in range(self.rows):
        for c in range(self.cols):
            if (
                self.horizontal[r][c]
                and self.horizontal[r + 1][c]
                and self.vertical[r][c]
                and self.vertical[r][c + 1]
            ):
                valid_completed.add((r, c))

self.completed = valid_completed

```

Do not remove your Task 1 fix from `_update_completed()` .

So `board.py` will now have both:

`</>` Plain text

```

_update_completed()
remove_line()

```

2. `game.py`

Add in `__init__` :

`</>` Python

```

self.history = []

```

Change the instructions to:

`</>` Python

```

print("Enter moves as H row col or V row col, or type UNDO to undo.")

```

Then add the `UNDO` handling **before**:

`</>` Python

```

parts = raw.split()

```

Use:

`</>` Python

```

if raw == "UNDO":
    if not self.history:
        print("Nothing to undo.")
        continue

    orientation, row, col, player, points = self.history.pop()
    self.board.remove_line(orientation, row, col)
    self.scores[player] -= points
    self.current = player
    print(f"Undid move {orientation} {row} {col} by Player {player + 1}.")
    continue

```

And after:

`</>` Python

```
newly_completed = completed_boxes(self.board, before)
```

add:

`</>` Python

```

self.history.append(
    (orientation, row, col, self.current, newly_completed)
)

```

STEP 5B — Test Task 2

First test the simple case:

`</>` Plain text

```

H 0 0
UNDO

```

Expected:

- The horizontal line disappears.
- Score remains 0-0 .
- Player 1's turn is restored.

Then test the important case:

`</>` Plain text

```

H 0 0
H 0 1

```

```
V 0 0
V 0 2
V 0 1
UNDO
```

Before UNDO :

```
</> Plain text

P1 = 2
P2 = 0
Turn = P1
```

After UNDO :

```
</> Plain text

P1 = 0
P2 = 0
Turn = P1
```

and the two `x` marks should disappear.

One important thing

Don't commit Task 2 yet.

First run these tests and make sure **Undo actually works**. If you get an error or unexpected turn/score, send me the output and we'll fix it before committing.

Once it works, we'll do the **separate Task 2 Git commit**, as required by your lab handout.

 Lab_4_VibeCoding_Student_handout

DONE NEXT

STEP 6 — Commit Task 2

Since you tested **Undo** and it is working, now save Task 2 as its own Git commit.

6.1 Check what changed

In the VS Code terminal:

```
</> Bash

git status
```

You should see:

`</>` Plain text

```
modified: board.py
modified: game.py
```

6.2 Review the changes

Run:

`</>` Bash

```
git diff
```

Make sure you see only the Task 2 changes:

- `remove_line()` in `board.py`
- `history` in `game.py`
- UNDO handling
- move-history recording

Your Task 1 fix should still be present.

6.3 Commit Task 2

Run these one at a time:

`</>` Bash

```
git add board.py game.py
```

Then:

`</>` Bash

```
git commit -m "Add undo last move feature"
```

Then:

`</>` Bash

```
git push
```

6.4 Check the commit history

Run:

 Bash

```
git log --oneline -4
```

You should now have separate commits similar to:

 Plain text

```
Add undo last move feature  
Fix box completion and turn handling  
...
```

That satisfies the handout's requirement for **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

STEP 7 — Task 3: Validation & Robustness

Now we move to Task 3.

Your README requires the program to safely handle:

1. malformed commands
2. invalid coordinates
3. repeated lines
4. moves after the board is complete
5. invalid input must **not change the board or score**

Your current code already handles **some** of these, so we should **inspect before changing anything**.

Use this prompt in the same LLM conversation:

Task 3 Prompt

① Original

② Write Like Me



I have completed Task 1 and Task 2 for Scenario 23 – Dots and Boxes.

Task 1 fixed the box-completion bug.

Task 2 added the Undo Last Move feature.

Now I need to complete Task 3 — Validation and Robustness.

The README requires the program to safely handle:

1. malformed commands
2. invalid coordinates
3. repeated lines
4. moves made after the board is already complete
5. invalid input must not corrupt the board or score

Please inspect my current board.py, rules.py, and game.py.

First identify which of these cases are already handled correctly and which are missing or unsafe.

Then propose the smallest modular changes needed to satisfy all Task 3 requirements.

Important:

- Do not remove or break the Task 1 bug fix.
- Do not remove or break the Task 2 Undo feature.
- Preserve the existing modular structure.
- Do not put all validation logic into main.py.
- Do not make unrelated changes.
- Explain which file each change belongs in before providing code.

Send me the LLM's response after you run that prompt. We'll check the proposed changes before applying them.

1. Assessment of Current Code vs Task 3 Requirements

Task 3 Requirement	Status	Module	Details
Malformed commands	Partially Handled	game.py	<code>len(parts) != 3</code> and <code>.isdigit()</code> checks handle most formatting, but negative coordinates (e.g., <code>-1</code>) pass <code>.isdigit()</code> checks improperly if represented with a minus sign or crash on non-digit checks.
Invalid coordinates	Handled	rules.py	<code>valid_move()</code> checks index bounds (<code>0 <= row <= board.rows</code> , etc.).

Task 3 Requirement	Status	Module	Details
Repeated lines	Handled	rules.py	<code>valid_move()</code> checks not <code>board.horizontal[row][col]</code> and not <code>board.vertical[row][col]</code> .
Moves after complete board	Missing / Unsafe	game.py / board.py	The main <code>while not self.board.is_complete()</code> : loop prevents input <i>while running standard turns</i> . However, if <code>board.add_line</code> or <code>valid_move</code> is called when the board is full, or if moves are processed without checking game state, there is no guard at the board/rules level.
Input error safety (no state corruption)	Handled	game.py	Invalid inputs trigger <code>continue</code> before touching <code>self.board.add_line()</code> or <code>self.history</code> , ensuring board, score, and history state remain uncorrupted.

2. File-by-File Breakdown of Required Changes

File 1: `rules.py`

- **Why change:** `rules.py` is the module responsible for validating move legality. It should explicitly ensure moves cannot be executed if the board is already complete.
- **What to add:** Add a check in `valid_move()` to return `False` immediately if `board.is_complete()` is `True`.

File 2: game.py

- **Why change:** game.py handles user input parsing and flow control. Negative numbers (e.g. H -1 0) cause str.isdigit() to evaluate to False (which displays "Row and column must be numbers"), but raw non-numeric strings or unexpected whitespace/case inputs need robust string handling without throwing uncaught exceptions.
- **What to add:** Enhance input parsing to safely handle integer conversion (using try/except ValueError instead of .isdigit()) to cover negative values cleanly.

3. Implementation Code**rules.py Update**

Python

```
def valid_move(board, orientation, row, col):
    # Reject moves if the board is already fully co
    if board.is_complete():
        return False

    if orientation not in {"H", "V"}:
        return False

    if orientation == "H":
        return (
            0 <= row <= board.rows
            and 0 <= col < board.cols
            and not board.horizontal[row][col]
        )
```

game.py Update

Python

```

from board import Board
from rules import valid_move, completed_boxes

class DotsAndBoxes:
    def __init__(self):
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]
        self.history = [] # Task 2: Undo history

    def run(self):
        print("Dots and Boxes")
        print("Enter moves as H row col or V row col")
        print("Rows and columns start at 0.")

```

4. Verification Checklist for Task 3

1. **Malformed Commands (h 0 , H 0 0 0 , F00):** Filtered out by `len(parts) != 3` without raising exceptions.
2. **Invalid / Negative Coordinates (H -1 0 , V 99 99):** Converted cleanly by `try...except ValueError` and rejected by `valid_move()`.
3. **Repeated Lines (H 0 0 twice):** `valid_move()` checks line availability and blocks repeated placements.
4. **Moves on Completed Board:** `valid_move()` checks `board.is_complete()` and rejects additional lines.
5. **State Preservation:** State is updated **only** after passing all validation steps, preventing score, turn, or history corruption.

Good. The Task 3 analysis is mostly correct, and the proposed changes are appropriate for your assignment. We can proceed carefully so we don't break Tasks 1 or 2.

STEP 7 — Apply Task 3

7.1 Change `rules.py`

Add the board-complete check at the **top** of `valid_move()` :

 Python

```
def valid_move(board, orientation, row, col):
    if board.is_complete():
        return False

    if orientation not in {"H", "V"}:
        return False

    if orientation == "H":
        return (
            0 <= row <= board.rows
            and 0 <= col < board.cols
            and not board.horizontal[row][col]
        )

    return (
        0 <= row < board.rows
        and 0 <= col <= board.cols
        and not board.vertical[row][col]
    )
```

7.2 Change `game.py`

Keep your Task 2 `history` and `UNDO` .

Replace the current `.isdigit()` section with the safer integer conversion:

 Python

```
parts = raw.split()

if len(parts) != 3:
    print("Invalid format. Use: H/V row col")
    continue

orientation, row_str, col_str = parts

try:
    row = int(row_str)
    col = int(col_str)
except ValueError:
    print("Row and column must be valid integers.")
    continue

if not valid_move(self.board, orientation, row, col):
    print("Invalid, out-of-bounds, or already-used move.")
    continue
```

And keep the existing move-processing code afterward.

7.3 Test all required cases

Run:

 Bash

```
python main.py
```

Test these **one at a time**.

A. Malformed command

 Plain text

```
FOO
H 0
H 0 0 0
```

They should be rejected without crashing.

B. Invalid coordinates

 Plain text

```
H -1 0
V 99 99
```

They should be rejected.

C. Repeated line

 Plain text

```
H 0 0
H 0 0
```

The second one should be rejected.

D. Undo still works

 Plain text

```
H 0 0
UNDO
```

The line should disappear and the state should be restored.

E. Task 1 still works

Plain text

```
H 0 0
H 0 1
V 0 0
V 0 2
V 0 1
```

You should still get 2 **boxes and another turn**.

Important

Don't commit yet.

After testing, run:

Bash

```
git status
```

and tell me whether all five tests worked. Then we'll do the **Task 3 commit** separately.

DONE NEXT

STEP 8 — Commit Task 3

Since you tested Task 3 successfully, now create the **separate Git commit**.

8.1 Check Git status

Run:

Bash

```
git status
```

You should see:

Plain text

```
modified: game.py
modified: rules.py
```

8.2 Review the changes

Run:

```
</> Bash
```

```
git diff
```

Confirm the changes are only for Task 3:

- safer integer parsing in `game.py`
- validation of completed board in `rules.py`
- Task 1 and Task 2 code is still present

8.3 Commit Task 3

Run:

```
</> Bash
```

```
git add game.py rules.py
```

Then:

```
</> Bash
```

```
git commit -m "Improve input validation and game robustness"
```

Then:

```
</> Bash
```

```
git push
```

8.4 Verify your commits

Run:

```
</> Bash
```

```
git log --oneline -5
```

You should now have separate commits similar to:

```
</> Plain text
```

```
Improve input validation and game robustness
Add undo last move feature
Fix box completion and turn handling
...
```

This follows the lab requirement for **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

STEP 9 — Task 4: Automated Tests

Now we are at the **last coding task**.

Your README requires tests for:

1. valid horizontal move
2. valid vertical move
3. invalid/repeated move
4. completion of a box
5. end-of-game condition

It also says to document important design decisions.

First, don't create the tests manually yet.

Use your LLM with this prompt:

Task 4 Prompt

① Original

② Write Like Me



I have completed Tasks 1, 2, and 3 for Scenario 23 – Dots and Boxes.

Task 1: Fixed the box-completion bug.

Task 2: Added Undo Last Move.

Task 3: Improved input validation and robustness.

Now I need to complete Task 4 — Testing and Quality.

The README requires automated tests covering:

1. a valid horizontal move
2. a valid vertical move
3. an invalid/repeated move
4. completion of a box
5. the end-of-game condition

Please inspect my current project structure and existing code before proposing the tests.

Requirements:

- Tests should test the existing modular code rather than rewriting the game.
- Do not break Tasks 1–3.
- Keep third-party dependencies out unless there is a clear need.
- Prefer Python's standard unittest framework if suitable.
- Create or expand an appropriate test file.
- Explain what each test verifies.
- Also identify the important design decisions from Tasks 1–4 that should be documented for the assignment.

First give me the proposed test file and explain where it should be placed. Do not make unrelated changes.

Send me the LLM's response.

Don't create the test file yet. I'll check the proposed tests with your existing project before you add them.

1. Test File Placement & Architecture

To keep the test suite clean and isolated from the main game code, create a new file named `test_game.py` in the root directory of your project (alongside `board.py`, `rules.py`, and `game.py`).

Using Python's standard `unittest` framework ensures zero external dependencies while covering all game scenarios required by the README.

2. Proposed Test Suite (`test_game.py`)

Python

```
import unittest
from board import Board
from rules import valid_move, completed_boxes
```

```
class TestDotsAndBoxes(unittest.TestCase):
```

```
    def setUp(self):
```

```
"""Sets up a standard 2x2 board before each
self.board = Board(rows=2, cols=2)
```

```
def test_valid_horizontal_move(self):
    """Verifies that a valid horizontal move is
    self.assertTrue(valid_move(self.board, "H"
```

3. Explanation of Each Test

1. **test_valid_horizontal_move :**

- Ensures `valid_move()` returns `True` for valid H coordinates.
- Confirms `add_line()` marks `self.horizontal[row][col] = True` .

2. **test_valid_vertical_move :**

- Ensures `valid_move()` returns `True` for valid V coordinates.
- Confirms `add_line()` marks `self.vertical[row][col] = True` .

3. **test_invalid_or_repeated_move :**

- Tests line repetition by attempting to draw H 0 0 twice.
- Checks out-of-bounds checks and invalid orientation flags ("X").

4. **test_completion_of_a_box :**

- Completes a box step-by-step and checks that `completed_boxes()` returns the correct new box count (1).
- Verifies coordinate (0, 0) is recorded inside `self.board.completed` .

5. **test_end_of_game_condition :**

- Fills every horizontal and vertical line on the 2 $\times$ 2 grid.
- Confirms `self.board.is_complete()` becomes `True` .
- Verifies `valid_move()` rejects subsequent moves on a completed board.

To run the test suite from your terminal, execute:

Bash

```
python -m unittest test_game.py
```

4. Key Design Decisions to Document for Assignment Submission

When writing your assignment report or README documentation, highlight these architectural decisions across Tasks 1–4:

- **Task 1 (Set-Difference Box Tracking):**
 - *Decision:* Used set-difference calculation `( len(board.completed - before) )` in `rules.py` instead of fixed single-box checks.
 - *Rationale:* Naturally handles moves that complete two boxes simultaneously without needing complex edge-case branching in the turn logic.
- **Task 2 (Stack-Based Move History & State Restoration):**
 - *Decision:* Implemented an explicit move history stack `( self.history )` in `game.py` alongside a clean `remove_line()` method in `board.py`.
 - *Rationale:* Keeps `board.py` focused on grid calculations and `game.py` focused on UI flow, ensuring UNDO completely restores scores, turns, and board line states without side effects.
- **Task 3 (Decoupled Input Validation & Defensive Checks):**
 - *Decision:* Combined `try...except ValueError` parsing in `game.py` with domain-level validation in `rules.py` (e.g., rejecting moves on full boards).
 - *Rationale:* Guarantees that invalid or malformed user inputs fail early before impacting score or board history state.
- **Task 4 (Zero-Dependency Modular Test Coverage):**
 - *Decision:* Employed Python's standard `unittest` framework to independently test core game components `( Board , rules )`.
 - *Rationale:* Verifies game mechanics programmatically without needing to mock console inputs or refactor `game.py`.

Yes. The proposed **Task 4 test suite is suitable** for your README requirements, and it uses Python's built-in `unittest`, so you don't need a third-party package.

STEP 9 — Create `test_game.py`

In the same folder where you have:

Plain text

```
board.py
rules.py
game.py
main.py
```

create a new file:

Plain text

```
test_game.py
```

Paste the complete test code from the LLM response into it.

Important

Don't modify `board.py` , `rules.py` , or `game.py` while creating the test file.

Your project should now look approximately like:

Plain text

```
Scenario-23-Dots-and-Boxes/
|
├─ main.py
├─ board.py
├─ rules.py
├─ game.py
└─ test_game.py      ← NEW
```

STEP 10 — Run the automated tests

In the terminal, make sure you're in the project folder and run:

Bash

```
python -m unittest test_game.py
```

Expected result

You want to see something like:

 Plain text

.....

Ran 5 tests in ...

OK

The five tests correspond directly to your README:

 Plain text

✓ Valid horizontal move

✓ Valid vertical move

✓ Invalid/repeated move

✓ Box completion

✓ End-of-game condition

The test suite also checks the Task 3 full-board validation.

If you get OK

Stop there. Don't commit yet.

Send me the terminal output showing:

 Plain text

Ran 5 tests

OK

Then I'll guide you through the final **Task 4 commit**, followed by the **two required 10-second videos and Chat/LLM history**, which are the final submission deliverables. The college handout confirms the Lab-4 repo should contain the videos, updated code, and exported chat history.

 Lab_4_VibeCoding_Student_handout

DONE NEXT

STEP 11 — Commit Task 4

Your automated tests are working, so now make the **separate Task 4 commit**.

11.1 Check Git status

Run:

 Bash

```
git status
```

You should see:

</> Plain text

```
modified: ...  
untracked: test_game.py
```

The modified files may depend on exactly what changed during Task 3.

11.2 Add the test file

</> Bash

```
git add test_game.py
```

Then commit:

</> Bash

```
git commit -m "Add automated tests for game rules"
```

Then push:

</> Bash

```
git push
```

11.3 Check your commit history

Run:

</> Bash

```
git log --oneline -5
```

You should now have **four separate task commits**, approximately:

</> Plain text

```
Add automated tests for game rules  
Improve input validation and game robustness
```

- Add undo last move feature
- Fix box completion and turn handling

That satisfies the requirement for separate commits for each task.  Lab_4_VibeCoding_Student_handout

STEP 12 — Check your final code

Before making the final video, **do not make any more feature changes.**

Your project should contain at least:

 Plain text

```
main.py
board.py
rules.py
game.py
test_game.py
```

And the completed work is:

Task	Completed
Task 1 — Bug fix	
Task 2 — Undo feature	
Task 3 — Validation	
Task 4 — Automated tests	

Now we have to finish the **submission deliverables.**

The handout says the Lab-4 repository needs **Videos, Updated Code, and Chat history exported as a document/PDF.**  Lab_4_VibeCoding_Student_handout

STEP 13 will be the videos

We need:

1. Before.mp4 — 10 seconds

- Original broken behavior
- The box-completion bug visible

2. After.mp4 — 10 seconds

- Fixed two-box completion
- Score/turn correct
- Undo feature working

Do not record the After video yet.

First tell me whether your **four Git commits are visible in** `git log --oneline` . Then I'll give you the exact 10-second **After video sequence** so you can record it cleanly.

First tell me whether your **four Git commits are visible in** `git log --oneline` WHERE IT IS

It is inside your project folder in VS Code / Terminal. You don't need to find a file called `git log` .

Do this:

1. Open your **Dots and Boxes project** in VS Code.
2. Open the terminal:
 - **VS Code → Terminal → New Terminal**
3. Make sure the terminal path is your project folder, something like:

</> Plain text

```
PS C:\Users\spoor\...\dots-and-boxes>
```

4. Type:

</> Bash

```
git log --oneline -5
```

5. Press Enter.

It will display your recent commits directly in the terminal, for example:

</> Plain text

```
abc1234 Add automated tests for game rules
def5678 Improve input validation and game robustness
ghi9012 Add undo last move feature
jkl3456 Fix box completion and turn handling
```

You don't need to open any file for this.

👉 Run `git log --oneline -5` and **send me a screenshot of what appears**. I'll check whether your 4 task commits are there.

```
C:\Users\spoor\scenario-23-dots-and-boxes>git log --oneline -5
fe5531d (HEAD -> main, origin/main, origin/HEAD) Add
automated tests and document design
a5bcd3e Improve input validation and robustness
33fbe59 Add undo last move feature
f32c96a Fix final move completion message
a0ad1a1 Updated scenario title
```

```
C:\Users\spoor\scenario-23-dots-and-boxes>
```

Yes — your Git history is visible. 

You have **4 relevant commits**, but notice the names:

 Plain text

```
fe5531d Add automated tests and document design
a5bcd3e Improve input validation and robustness
33fbe59 Add undo last move feature
f32c96a Fix final move completion message
```

So your work is committed and pushed to `origin/main` because you have:

 Plain text

```
HEAD -> main, origin/main
```

One important point

Your Task 1 commit is named:

```
Fix final move completion message
```

instead of the exact message I previously suggested. **That's completely okay**. The important thing is that the Task 1 bug fix was committed.

And your Task 4 commit:

```
Add automated tests and document design
```

also looks good.

✓ Git part is DONE

Now we move to the **final submission videos**.

You need:

- **Before video:** 10 seconds, showing the original bug
- **After video:** 10 seconds, showing the fixed game/features

The handout specifically requires the before/after videos.  [Lab_4_VibeCoding_Student_handout](#)

Next: I'll give you the exact moves to perform for the 10-second After video.

YES

Great. 👍 For the **After video**, we want to show that your completed Tasks actually work.

🎥 After Video — 10 seconds

Open your terminal in:

 Plain text

```
C:\Users\spoor\scenario-23-dots-and-boxes
```

Run:

 Bash

```
python main.py
```

Then record the screen and quickly enter these moves:

 Plain text

```
H 0 0
H 0 1
V 0 0
V 0 2
V 0 1
```

The final move should complete **2 boxes**.

You should see something like:

 Plain text

Player 1 completed 2 box(es) and plays again.

and the board should show **two X's**, with:

</> Plain text

Scores: P1=2 P2=0 | Turn: P1

Then show Undo

Immediately type:

</> Plain text

UNDO

You should see:

</> Plain text

Undid move V 0 1 by Player 1.

The completed boxes should disappear and the score should return accordingly.

⚠ Important

You don't need to demonstrate every validation case in the 10-second video. The assignment asks for a **10-second after video**, so keep it short. The automated tests in `test_game.py` document the other cases.

Save it as:

</> Plain text

After.mp4

Don't upload it to GitHub yet. First make sure the recording clearly shows the fixed behavior.

DONE

Perfect. ☒ After.mp4 is done.

Now only the **submission organization + LLM chat history** remains.

Next: Create the Lab-4 folder in your personal GitHub

We will eventually arrange it like:

</> Plain text

```
Lab-4/  
├── Videos/  
│   ├── Before.mp4  
│   └── After.mp4  
│  
├── Updated_Code/  
│   └── scenario-23-dots-and-boxes/  
│       ├── main.py  
│       ├── board.py  
│       ├── rules.py  
│       ├── game.py  
│       ├── test_game.py  
│       └── other project files  
│  
└── Chat_History/  
    └── LLM_Conversation.pdf
```

The handout requires the **videos**, **updated code**, and **exported LLM/chat history**.

 Lab_4_VibeCoding_Student_handout

Don't create/upload everything at once. We'll do it one step at a time.

First, open your personal GitHub repository

Go to your **personal GitHub repo** where you need to submit Lab 4.

Tell me when you have the repository page open, and I'll tell you exactly what to click to create Lab-4 .